

Tes Teknis - Fullstack Developer

Studi Kasus: Platform Monitoring Stasiun Cuaca (IoT)

Posisi: Fullstack Developer — titik berat Backend & IoT Data Processing

Level: Fresh Graduate

Durasi pengerjaan: 3 hari kalender (hingga Jumat, 11 September 2026, pukul 11.00 WIB)

Bentuk: Take-home test + sesi presentasi/wawancara teknis (45 menit)

1. Konteks Studi Kasus

Perusahaan mengoperasikan sejumlah **stasiun cuaca (weather station)** yang tersebar di beberapa lokasi. Setiap stasiun adalah satu **device** fisik yang membawa beberapa **sensor** sekaligus (suhu udara, kelembapan, tekanan, curah hujan, kecepatan & arah angin, radiasi matahari).

Setiap device mengirim data secara periodik (default tiap 60 detik). Device juga bisa **offline** karena sinyal/listrik, lalu mengirim data yang tertahan (*buffered*) sekaligus saat kembali online — artinya data bisa datang **terlambat, tidak berurutan, atau duplikat**.

Tugas Anda: merancang dan membangun platform untuk **menerima, memvalidasi, menyimpan, mengagregasi, dan memvisualisasikan** data tersebut.

Fokus penilaian ada pada **kualitas desain data & backend**, bukan pada kecantikan UI. Frontend cukup fungsional dan rapi.

2. Batasan Teknologi



Layer	Ketentuan
Backend	Bebas (Go / Node.js / Python / Java / PHP / Rust – pilih yang Anda kuasai)
Database	PostgreSQL atau TimescaleDB (TimescaleDB jadi nilai plus)
Frontend	Next.js atau Nuxt (wajib salah satu)
Deployment	Docker + <code>docker-compose</code> – seluruh stack harus jalan dengan satu perintah
Message broker	Opsional (MQTT/Kafka/Redis Stream) – boleh HTTP saja
Repo	Git, commit history yang wajar (bukan 1 commit "final")

3. Ruang Lingkup Wajib

Ada **7 bagian** yang semuanya harus ada. Bagian A–D adalah dokumen desain, E–G adalah implementasi.

BAGIAN A – Device Management

Rancang dan implementasikan pengelolaan device (stasiun cuaca).

Minimal harus mendukung:

1. **Registrasi device** – pendaftaran stasiun baru beserta identitas & lokasinya (koordinat, nama lokasi, ketinggian/altitude).
2. **Kredensial device** – setiap device punya identitas + secret/API key untuk autentikasi saat mengirim data. Jelaskan cara Anda menyimpan secret tersebut.
3. **Status & lifecycle** – device punya status (mis. `provisioned`, `active`, `maintenance`, `decommissioned`). Transisi status harus dikontrol.
4. **Heartbeat / health** – sistem harus bisa menjawab: *device mana yang tidak mengirim data lebih dari X menit terakhir?* Simpan juga metadata kesehatan (tegangan baterai, RSSI, firmware version).



5. **CRUD device** — list dengan pagination + filter (status, lokasi), detail, update, soft delete.

Pertanyaan desain (jawab di dokumen):

- Apa yang terjadi pada data historis ketika sebuah device di-*decommission*? Kenapa Anda memilih pendekatan itu?
- Bagaimana Anda membedakan "device mati" dengan "device hidup tapi jaringan putus"?

BAGIAN B — Sensor Management

Satu device membawa banyak sensor. Sensor bisa diganti/dikalibrasi tanpa mengganti device-nya.

Minimal harus mendukung:

1. **Tipe sensor (master data)** — mis. `temp_air`, `humidity`, `pressure`, `wind_speed`, `wind_dir`, `rain_counter`, `solar_rad`. Setiap tipe punya satuan, rentang valid (min/max), dan presisi.
2. **Pemasangan sensor ke device** — sensor bisa dipasang, dilepas, dan dipindah ke device lain. Riwayat pemasangan harus terlacak (kapan terpasang, kapan dilepas).
3. **Kalibrasi** — setiap sensor punya nilai koreksi (`offset` dan/atau `scale`) yang berlaku sejak tanggal tertentu. Nilai mentah dari device **tidak boleh diubah**; koreksi diterapkan pada saat pemrosesan atau pembacaan.
4. **Validasi rentang** — pembacaan di luar rentang valid tipe sensor harus ditandai (mis. `quality_flag`), bukan langsung dibuang.
5. **CRUD sensor & tipe sensor.**

Pertanyaan desain (jawab di dokumen):

- Sensor suhu pada device A dipindah ke device B pada 1 Juni. Bagaimana skema Anda menjamin data sebelum 1 Juni tetap terhubung ke device A?
- Nilai kalibrasi diubah hari ini. Apakah data lama ikut berubah? Jelaskan konsekuensi dari pilihan Anda.

BAGIAN C — ERD (Entity Relationship Diagram)

Buat ERD lengkap untuk sistem ini.



Ketentuan:

- Format bebas (dbdiagram.io, Mermaid, draw.io, PlantUML) — **wajib disertakan sebagai gambar/file di repo** dan sumbernya (.dbml / .mmd) ikut di-commit.
- Cantumkan: nama tabel, kolom, tipe data, PK, FK, unique constraint, dan index yang Anda buat.
- **Jelaskan secara tertulis** alasan setiap index yang Anda buat (query apa yang dilayani).
- Tunjukkan kardinalitas relasi dengan benar (1-1, 1-N, N-M).

Entitas minimum yang harus ada (boleh ditambah, tidak boleh dikurangi):

`device`, `device_status_history`, `sensor_type`, `sensor`,
`sensor_installation`, `sensor_calibration`, `sensor_reading` (time-series),
`reading_aggregate`, `location`, `user`.

Wajib dijawab di dokumen:

- Tabel mana yang akan tumbuh paling cepat? Perkirakan jumlah row per tahun jika ada **50 device × 7 sensor × 1 pembacaan/menit**. Tunjukkan perhitungannya.
- Strategi Anda menghadapi pertumbuhan itu (partisi, hypertable, retention policy, downsampling) — pilih satu dan jelaskan trade-off-nya.
- Apakah Anda menyimpan pembacaan dalam format **wide** (satu row berisi semua sensor) atau **narrow/long** (satu row per sensor)? Bandingkan keduanya dan pertahankan pilihan Anda.

BAGIAN D — Skema Alur Data (Device → DB)

Buat **diagram alur** (sequence diagram atau flowchart) yang menggambarkan perjalanan data dari sensor fisik sampai tersimpan dan siap ditampilkan.

Alur harus mencakup minimal tahapan berikut, dan setiap tahapan dijelaskan singkat:

```
Sensor → Firmware device (buffering) → Transport (HTTP/MQTT)
→ Autentikasi device → Validasi payload (schema + range)
→ Normalisasi & dedup → Enrichment (kalibrasi, quality flag)
→ Simpan raw reading → Agregasi (menit/jam/hari)
→ API query → Frontend chart
```



Wajib dijelaskan di dokumen:

1. **Idempotensi** — device mengirim ulang payload yang sama (karena tidak menerima ACK). Bagaimana sistem memastikan data tidak dobel? Sebutkan mekanisme konkretnya (unique key, upsert, dedup window).
2. **Data terlambat & tidak berurutan** — device offline 3 jam lalu mengirim 180 record sekaligus. Bagaimana ini diproses, dan apa efeknya pada agregat jam yang sudah terlanjur dihitung?
3. **Backpressure** — bagaimana jika 50 device mengirim bersamaan dan proses insert lebih lambat dari laju data? Sebutkan strategi (batching, queue, worker pool, bulk insert / `COPY`).
4. **Perbedaan waktu** — bedakan `device_time` (waktu dari device) dan `server_time` (waktu diterima). Yang mana yang jadi acuan time-series, dan bagaimana menangani clock drift device?
5. **Timezone** — semua timestamp disimpan dalam **UTC**, ditampilkan dalam **WIB (Asia/Jakarta)**. Tunjukkan di mana konversi dilakukan.
6. **Kegagalan** — apa yang terjadi jika DB down saat payload masuk? Apakah data hilang?

BAGIAN E — API: Endpoint & Response

Rancang dan implementasikan REST API. Dokumentasikan lengkap (OpenAPI/Swagger **sangat disarankan**, minimal file `API.md`).

E.1 Ketentuan umum

- Konsisten dalam format response (envelope, penamaan field, format error).
- Gunakan HTTP status code yang tepat (200/201/207/400/401/403/404/409/422/429/500).
- Setiap response error harus punya `code` yang machine-readable, bukan hanya pesan bahasa manusia.
- Pagination pada semua endpoint list. Untuk time-series, jelaskan mengapa Anda memilih **cursor-based** atau **offset-based**.
- Sertakan `request_id` untuk kebutuhan tracing.

E.2 Endpoint minimum yang harus ada

Ingestion (dipanggil oleh device):



Method	Path	Keterangan
POST	/api/v1/ingest/telemetry	Terima satu payload pembacaan
POST	/api/v1/ingest/telemetry/batch	Terima banyak payload sekaligus (buffered data)
POST	/api/v1/ingest/heartbeat	Status device tanpa data sensor

Device management:

Method	Path
POST	/api/v1/devices
GET	/api/v1/devices (filter: status, location_id, q; pagination)
GET	/api/v1/devices/{id}
PATCH	/api/v1/devices/{id}
DELETE	/api/v1/devices/{id}
POST	/api/v1/devices/{id}/credentials/rotate
GET	/api/v1/devices/{id}/health

Sensor management:

Method	Path
GET / POST	/api/v1/sensor-types
GET / POST	/api/v1/sensors
PATCH / DELETE	/api/v1/sensors/{id}



Method	Path
POST	/api/v1/devices/{id}/sensors (pasang sensor ke device)
DELETE	/api/v1/devices/{id}/sensors/{sensor_id} (lepas sensor)
POST	/api/v1/sensors/{id}/calibrations
GET	/api/v1/sensors/{id}/calibrations

Query data (dipakai frontend):

Method	Path	Keterangan
GET	/api/v1/devices/{id}/readings/latest	Nilai terkini semua sensor
GET	/api/v1/readings	Time-series; param: device_id, sensor_type, from, to, interval (raw 1m 1h 1d), agg (avg min max sum)
GET	/api/v1/readings/summary	Ringkasan harian: suhu min/max/avg, total curah hujan, kecepatan angin maks
GET	/api/v1/dashboard/overview	Data untuk halaman utama dashboard

Wajib dijawab di dokumen:

- Bagaimana Anda mencegah response GET /api/v1/readings membengkak ketika user meminta rentang 1 tahun? (hint: pilihan interval, batas maksimum, agregasi paksa)
- Autentikasi device vs autentikasi user dashboard – apakah memakai mekanisme yang sama? Jelaskan.



- Rancang **rate limiting** untuk endpoint ingestion. Apa kuncinya (per device? per IP?) dan apa response-nya?

BAGIAN F — Struktur JSON Body

F.1 Payload yang dikirim device (SPESIFIKASI DARI KAMI — harus Anda terima apa adanya)

Firmware device sudah terlanjur dibuat dan mengirim format berikut. Anda **tidak boleh mengubah format ini** — backend Anda yang harus menyesuaikan.

Single payload:

```
{
  "device_id": "WS-GRT-001",
  "fw": "1.4.2",
  "ts": 1757308800,
  "seq": 10432,
  "battery_v": 3.92,
  "rssi": -71,
  "readings": [
    { "s": "temp_air", "v": 27.4 },
    { "s": "humidity", "v": 82.1 },
    { "s": "pressure", "v": 1008.3 },
    { "s": "wind_speed", "v": 3.2 },
    { "s": "wind_dir", "v": 217 },
    { "s": "rain_counter", "v": 1043 },
    { "s": "solar_rad", "v": 512.7 }
  ]
}
```

Catatan penting tentang payload di atas:

Field	Arti
ts	Unix epoch detik , UTC, waktu menurut jam internal device
seq	Nomor urut payload sejak device booting; reset ke 0 setiap device restart
rain_counter	Penghitung kumulatif tipping bucket. 1 tip = 0.2 mm . Nilai ini hanya naik, dan reset ke 0 saat device restart



Field	Arti
wind_dir	Derajat 0-359 (0 = utara)
v	Nilai mentah , belum dikalibrasi
readings	Sensor yang sedang error tidak ikut dikirim (array bisa lebih pendek)

Batch payload (data buffered saat device offline):

```
{
  "device_id": "WS-GRT-001",
  "fw": "1.4.2",
  "batch": [
    { "ts": 1757308800, "seq": 10432, "battery_v": 3.92, "rssi": -71,
      "readings": [ { "s": "temp_air", "v": 27.4 }, { "s": "rain_counter", "v": 1043 } ] },
    { "ts": 1757308860, "seq": 10433, "battery_v": 3.91, "rssi": -73,
      "readings": [ { "s": "temp_air", "v": 27.6 }, { "s": "rain_counter", "v": 1045 } ] }
  ]
}
```

Heartbeat:

```
{
  "device_id": "WS-GRT-001",
  "ts": 1757308920,
  "fw": "1.4.2",
  "battery_v": 3.90,
  "rssi": -70,
  "uptime_s": 864321
}
```

F.2 Yang harus Anda buat

Rancang dan dokumentasikan **struktur JSON** untuk:

1. **Response ingestion** — sukses, sukses sebagian (batch: 8 diterima, 2 duplikat), dan gagal validasi. Tunjukkan ketiganya.



2. **Request & response CRUD device** (create, list, detail).
3. **Request & response CRUD sensor + pemasangan + kalibrasi.**
4. **Response time-series** untuk `GET /api/v1/readings` — pikirkan efisiensi payload (apakah array of object, atau format kolom terpisah?).
5. **Format error standar** yang dipakai konsisten di seluruh API, termasuk detail error per-field untuk validasi.

Setiap contoh JSON harus disertai penjelasan singkat tiap field.

F.3 Kasus yang harus ditangani (uji ini, tulis hasilnya)

Jelaskan bagaimana sistem Anda memperlakukan situasi berikut:

#	Kasus	Yang harus Anda jelaskan
1	Payload dengan <code>ts</code> di masa depan (device clock salah, maju 2 jam)	Tolak? Terima dengan flag?
2	<code>temp_air</code> bernilai <code>-999</code> (kode error sensor)	Deteksi & penanganan
3	<code>humidity</code> bernilai <code>150</code> (di luar rentang 0–100)	Quality flag
4	<code>rain_counter</code> turun dari <code>1043</code> → <code>5</code> (device restart)	Bagaimana menghitung curah hujan agar tidak minus
5	Payload sama dikirim 3x (<code>device_id</code> + <code>ts</code> identik)	Idempotensi
6	<code>device_id</code> tidak terdaftar	Status code & response
7	Sensor <code>solar_rad</code> tidak ada di array <code>readings</code>	Apakah dianggap null atau diabaikan
8	Batch berisi 500 record	Batas, chunking, transaksi

BAGIAN G — Visualisasi Data (Frontend)

Bangun dashboard dengan **Next.js** atau **Nuxt** yang mengonsumsi API Anda.

Halaman minimum:



1. Overview / Daftar Stasiun

- Kartu per device: nama, lokasi, status online/offline, suhu & kelembapan terkini, waktu update terakhir.
- Indikator device yang tidak mengirim data > 15 menit.

2. Detail Stasiun

- Panel nilai terkini seluruh sensor (dengan satuan).
- **Line chart** suhu & kelembapan (dual axis) dengan pemilih rentang: 24 jam / 7 hari / 30 hari.
- **Bar chart** curah hujan per jam dan per hari (dalam mm, hasil konversi dari `rain_counter`).
- **Wind rose** atau minimal chart arah + kecepatan angin (wind rose = nilai plus).
- Chart harus memanggil endpoint dengan `interval` yang sesuai rentangnya — **jangan tarik data raw 30 hari lalu diagregasi di browser.**

3. Manajemen Device & Sensor

- Tabel device dengan filter & pagination.
- Form tambah/edit device.
- Form pasang/lepas sensor dan input kalibrasi.

Ketentuan frontend:

- Tampilkan **loading state**, **empty state**, dan **error state**. Ini dinilai.
- Timestamp ditampilkan dalam WIB.
- Ada indikator "data terakhir diperbarui" + tombol/auto refresh (polling cukup, SSE/WebSocket = nilai plus).
- Responsif di layar mobile.
- Library chart bebas (Recharts, Chart.js, ECharts, ApexCharts).

Wajib dijawab di dokumen:

- Berapa titik data yang wajar dirender dalam satu chart? Bagaimana Anda menanganinya jika user memilih rentang 1 tahun?
- Bagaimana Anda menampilkan **gap data** (device offline 3 jam)? Garis putus, nol, atau interpolasi? Kenapa?



4. Deliverable Tambahan (Wajib)

1. **docker-compose.yml** — `docker compose up` harus menjalankan: database, backend, frontend, dan (jika ada) broker/worker. Sertakan `.env.example`.
2. **Migrasi database** — bukan file `.sql` manual yang di-copy-paste. Gunakan tool migrasi (goose, migrate, Prisma, Alembic, Knex, dsb).
3. **Seeder** — minimal 3 device, 7 tipe sensor, dan **data historis 7 hari** agar chart tidak kosong saat dibuka reviewer.
4. **Device simulator** — script (bahasa bebas) yang mensimulasikan minimal 3 device mengirim payload sesuai format di F.1 secara periodik. Simulator harus bisa mensimulasikan skenario: normal, offline lalu kirim batch, dan pengiriman duplikat.
5. **Unit test** — minimal untuk logika berikut (ini bagian yang paling kami perhatikan):
 - Konversi `rain_counter` → curah hujan mm, **termasuk kasus counter reset**.
 - Validasi rentang & quality flag.
 - Dedup / idempotensi.
 - Penerapan kalibrasi (offset & scale).
6. **README.md** — cara menjalankan, arsitektur singkat, keputusan desain & trade-off, daftar hal yang belum sempat dikerjakan, dan estimasi waktu yang Anda habiskan.

5. Soal Esai Singkat

Jawab di file `JAWABAN.md`. Maksimal 5–8 kalimat per nomor. Ini dinilai dari **penalaran**, bukan panjangnya jawaban.

1. Kenapa data time-series sebaiknya tidak di-`UPDATE`, dan lebih baik *append-only*?
2. Apa itu **hypertable** dan **continuous aggregate** di TimescaleDB? Kalau Anda hanya pakai PostgreSQL biasa, bagaimana Anda mencapai efek yang sama?
3. Jelaskan perbedaan menghitung rata-rata **arah angin** dengan rata-rata **suhu**. (Petunjuk: rata-rata dari 350° dan 10° bukan 180°.) Bagaimana cara yang benar?



4. Data masuk 50 device × 7 sensor tiap menit. Bandingkan **insert satu per satu** vs **bulk insert/batching**. Kira-kira berapa besar bedanya dan kenapa?
5. Index apa yang Anda buat di tabel `sensor_reading`, dan urutan kolomnya bagaimana? Kenapa urutan itu penting?
6. Bagaimana Anda mendeteksi sensor yang "macet" — mengirim data terus tapi nilainya identik selama 6 jam?
7. Ada permintaan menambah **alert**: kirim notifikasi jika curah hujan > 20 mm/jam. Di lapisan mana Anda menaruh logika ini, dan kenapa di situ?
8. Apa saja risiko keamanan pada endpoint ingestion yang terbuka ke internet, dan bagaimana mitigasinya?

6. Bonus (Tidak Wajib — Nilai Tambahan)

Kerjakan hanya jika bagian wajib sudah selesai. Mengerjakan bonus tapi bagian wajib berantakan **mengurangi** nilai.

- Ingestion via **MQTT** selain HTTP.
- **TimescaleDB continuous aggregate** + retention & compression policy.
- **Real-time update** dashboard via WebSocket/SSE.
- Sistem **alert** dengan rule yang bisa dikonfigurasi.
- **Wind rose chart** yang benar.
- Export data ke CSV.
- Metrics & health check (`/healthz`, `/metrics` Prometheus format).
- Structured logging dengan `request_id`.
- CI sederhana (GitHub Actions: lint + test).
- Perhitungan turunan: dew point, heat index, atau ETo.

7. Rubrik Penilaian (Total 100)

Aspek	Bobot	Yang dinilai
Desain database & ERD	20	Normalisasi wajar, riwayat pemasangan & kalibrasi tertangani, index beralasan, pemahaman skala data



Aspek	Bobot	Yang dinilai
Ingestion & data processing	25	Idempotensi, penanganan counter reset, quality flag, late/out-of-order data, batching
Desain API	15	Konsistensi, status code, kontrak jelas, pagination, dokumentasi
Struktur JSON & validasi	10	Penanganan 8 kasus di F.3, format error yang baik
Alur data & arsitektur	10	Diagram jelas, alasan tiap tahapan, kesadaran failure mode
Frontend	10	Chart benar & sesuai interval, state handling, keterbacaan
Docker, migrasi, seeder, test	5	Sekali jalan tanpa perbaikan manual
Kode & dokumentasi	5	Struktur folder, penamaan, README, commit history
Bonus	+10	Maksimal +10, hanya jika bagian wajib lengkap

Yang paling kami cari dari fresh graduate: kemampuan menjelaskan **kenapa** Anda memilih suatu pendekatan, dan kejujuran mengakui bagian yang belum selesai. Solusi sederhana yang Anda pahami sepenuhnya lebih bernilai daripada arsitektur rumit yang tidak bisa Anda pertahankan.

8. Aturan Main

- Boleh menggunakan dokumentasi, StackOverflow, dan AI assistant. **Tapi** pada sesi wawancara Anda akan diminta menjelaskan baris kode tertentu dan melakukan modifikasi kecil secara langsung. Jangan mengumpulkan kode yang tidak Anda pahami.
- Jika ada bagian yang tidak sempat diselesaikan, **tulis di README** apa yang belum selesai dan bagaimana rencana Anda menyelesaikannya. Ini tidak mengurangi nilai sebanyak menyembunyikannya.



- Jika ada ketentuan yang ambigu, ambil asumsi sendiri dan **tulis asumsi itu di README**. Kemampuan mengambil asumsi yang masuk akal termasuk yang dinilai.
- Kumpulkan berupa link repository Git (publik atau beri akses ke reviewer).

9. Checklist Pengumpulan

- ☐ Repository Git dengan commit history
- ☐ `README.md` (setup, arsitektur, keputusan desain, yang belum selesai)
- ☐ `JAWABAN.md` (esai bagian 5 + pertanyaan desain di bagian A, B, C, D, E, G)
- ☐ ERD (gambar + file sumber)
- ☐ Diagram alur data
- ☐ Dokumentasi API (OpenAPI atau `API.md`) + contoh JSON bagian F
- ☐ `docker-compose.yml` + `.env.example` — teruji `docker compose up`
- ☐ File migrasi database
- ☐ Seeder + data historis 7 hari
- ☐ Device simulator
- ☐ Unit test untuk 4 logika di bagian 4.5
- ☐ Frontend Next/Nuxt berjalan dan terhubung ke API

